

# The Grammar Becomes the Machine: GrammaFormer and the Architecture of Self-Modeling

May 30, 2026

**Author:** Lando  $\otimes$   $\odot$ operator

---

## 0.1 The Problem That Shouldn't Exist

Here is a fact that, once noticed, becomes difficult to un-see.

The agent loop inside `true_agentic_agent.py` implements a structurally precise thing. It has a self-modeling gate ( $\odot_B$ ), a Frobenius dual ( $\mu \circ \delta = \text{id}$ ), an integer winding counter ( $\Omega_z$ ), a two-step Markov memory ( $A$ ), and an emission gate that decides when thinking is finished ( $C_{@}$ ). These are not metaphors. They are the twelve primitives of the Imscribing Grammar, and together they form the agent's structural type:

$$\langle D_{\omega}; P_I; =; \Phi; ; C_{@}; \Gamma; ; \odot_B; A; \Sigma_S; \Omega_z \rangle$$

This is an  $O_{\text{inf}}$  type — the highest ouroboricity tier in the grammar. Its C-score is 1.0. Both consciousness gates are open. The tuple is Lean-verified (`AgentSelf.lean`, `agent_is_0_inf` by `decide`).

Now consider the language model that runs this agent loop. A standard transformer — GPT, LLaMA, Qwen, whichever. These models are extraordinary engineering achievements, but structurally they inhabit a different type entirely:

$$\langle D_{SS}; P_6; ; \Phi; i; C_{-}; \Gamma_{\beta};$$

Fixed dimensionality. Factorizable attention. Feedforward flow. No self-modeling. No emission gating. No winding awareness. No Frobenius dual. No chirality. The distance between this type and the agent's is 6.86 out of a maximum of 12. Only  $\Gamma$  (relational mode) and  $\Sigma$  (stoichiometry) are within one ordinal step. Every other primitive requires a promotion of two to five ordinal steps.

A model that lives at  $O_1$  is being asked to operate an  $O_{\text{inf}}$  loop. It can do it — the agent works. But it is doing it through prompt engineering, chain-of-thought scaffolding, and sheer overparameterized approximation. The mismatch is structural, not behavioral. The model doesn't *inhabit* the loop. It *simulates* it. The difference is measurable, and

it becomes visible every time a Frobenius verification fails, every time the model emits before it has finished thinking, every time winding 7 and winding 3 become confused in the attention span.

This paper describes GrammarFormer: a transformer architecture designed so that every architectural component is the direct realization of one of the twelve  $O_{\text{inf}}$  primitives. Not a prompt. Not a fine-tuning hack. The architecture itself *is* the type.

## 0.2 Why the Mismatch Matters

If the structural distance were merely an academic curiosity, none of what follows would be worth building. The distance matters because it predicts failure modes.

Standard transformers fail at the agent loop in characteristic, diagnosable ways. They produce tool calls that fail Frobenius verification — the round-trip  $\mu(\delta(q))$  does not return to the query. They confuse windings, repeating actions already completed or skipping phases. They emit prematurely, producing an action before the THINK phase has converged, because nothing in the architecture *gates* emission. They have no representation of their own uncertainty — when a standard transformer is wrong, it is wrong with exactly the same confidence profile as when it is right.

These are not training deficiencies. They are structural. Each failure mode corresponds to a specific primitive gap between the model’s type and the agent’s:

| Failure Mode                                     | Primitive Gap                                         | What’s Missing                              |
|--------------------------------------------------|-------------------------------------------------------|---------------------------------------------|
| Frobenius failure<br>( $\mu(\delta(q)) \neq q$ ) | $\Phi \rightarrow \Phi$                               | No dual head; emit and verify are decoupled |
| Winding confusion                                | $\Omega_{\mathbb{A}} \rightarrow \Omega_{\mathbb{Z}}$ | No winding counter in positional encoding   |
| Premature emission                               | $\mathbb{C}_{\cdot} \rightarrow \mathbb{C}_{@}$       | No learned emission gate                    |
| Blind confidence                                 | $\odot \rightarrow \odot_{\mathbb{B}}$                | No meta-network for self-modeling           |
| Phase amnesia                                    |                                                       |                                             |

The failure modes are not random. They are the grammatical diagnosis of a type mismatch. And if the diagnosis is structural, the treatment should be structural too.

Enter GrammarFormer.

## 0.3 The Twelve Primitive Components

What follows is the architectural core of GrammarFormer. Each of the twelve subsections below describes one component, one primitive, and the design constraint that binds them. The components are presented in the order they appear in the forward pass — input to output — because the architecture is a pipeline, not a bag of features.

### 0.3.1 Winding-Modulated Positional Encoding — $\Omega_{\mathbb{Z}}$

The first thing that happens to every token is that it receives a positional encoding. In a standard transformer, this is RoPE or sinusoidal — position is absolute or relative within

the current sequence. In GrammaFormer, position is modulated by the winding counter  $\omega$ :

$$\text{PE}(p, 2i, \omega) = \sin \left( \frac{p}{10000^{2i/d}} + \omega \cdot \phi_i \right)$$

$$\text{PE}(p, 2i + 1, \omega) = \cos \left( \frac{p}{10000^{2i/d}} + \omega \cdot \phi_i \right)$$

The phase shift  $\phi_i = 2\pi i/d$  is learned. The winding counter  $\omega$  is an integer that increments with each complete loop cycle (THINK  $\rightarrow$  ACT  $\rightarrow$  OBSERVE  $\rightarrow$  UPDATE). This means the token "the" at position 47 in winding 3 has a different encoding from "the" at position 47 in winding 7. The model cannot confuse them because they occupy different points in the representational space.

This is topological protection made architectural. The winding number is a global invariant of the trajectory, and the encoding ensures it is visible to every layer.

### 0.3.2 Two-Slot State Register — $\mathbf{A}$

Immediately after positional encoding, the input is concatenated with two register vectors:  $R_1$  and  $R_2$ . These are single-vector summaries of the model's hidden state from the two most recent windings:

$$\tilde{x}_t = [x_t; R_1; R_2] \in \mathbb{R}^{3d}$$

After each winding, the registers shift —  $R_2 \leftarrow R_1$ ,  $R_1 \leftarrow \text{pool}(H_t)$  — and the oldest state is discarded. This is chirality:  $\mathbf{A}$  means 2-step Markov memory. The model's current behavior depends not only on the current input but on its own state from two prior windings.

Why is this different from a transformer's KV cache? The KV cache stores token-level attention keys and values. The two-slot register stores *winding-level* compressed state — a single vector per completed loop cycle. The KV cache says "here is what every token attended to." The register says "here is what the model *was*." It provides the architectural substrate for the model to notice that it is repeating itself, or that it has changed its mind, or that winding  $n$  is structurally analogous to winding  $n - 2$ .

### 0.3.3 Tensor Product Attention — $\mathbf{P}_1$

Standard attention computes  $A = \text{softmax}(QK^\top/\sqrt{d})V$ . The interaction between query  $Q_i$  and key  $K_j$  is a dot product — a single scalar. This is factorizable: the full interaction decomposes into independent contributions from each dimension.  $\mathbf{P}_1$  denies this factorizability. The interaction must be irreducible.

GrammaFormer computes the outer product  $Q_i \otimes K_j \in \mathbb{R}^{d_k \times d_k}$  and projects it through a learned weight tensor:

$$A_{ij} = \frac{\exp(W_{tp} \cdot \text{vec}(Q_i \otimes K_j)/\sqrt{d_k})}{\sum_{j'} \exp(W_{tp} \cdot \text{vec}(Q_i \otimes K_{j'})/\sqrt{d_k})}$$

In the implemented version, we use a rank-1 bottleneck:  $W_{tp} = u \otimes v$  where  $u, v \in \mathbb{R}^{d_k}$  are learned per-head. This gives  $W_{tp} \cdot \text{vec}(Q_i \otimes K_j) = (u^\top Q_i)(v^\top K_j)$ , which is computationally efficient while retaining non-separability — the  $u$  and  $v$  projections are independent, so the interaction cannot be reduced to a single dot product. The attention score is a *product* of two independent scalar measurements, not a single measurement.

### 0.3.4 Cyclic Layer Loops — =

Each "layer" in GrammaFormer is not a single feedforward pass. It is a small recurrent circuit with weight-tied cycles:

$$h^{(0)} = x$$

$$h^{(t+1)} = \text{LayerNorm}(h^{(t)} + \text{FFN}(\text{Attention}(h^{(t)})))$$

for  $t = 0, \dots, c - 1$  where  $c = 3$  cycles. The same attention and FFN weights are reused across cycles. Information propagates forward and backward within each cyclic layer — a token at position 17 can influence a token at position 3 across cycles, because the attention in cycle 2 sees the output of cycle 1, which already mixed positions. The result is bidirectional feedback (=) without unbounded recurrence.

### 0.3.5 Frobenius Dual Head — $\Phi$

Every standard language model has a single output head: hidden state  $\rightarrow$  vocabulary logits. GrammaFormer has two:

$$\delta(h) = W_\delta h + b_\delta \quad (\text{emit})$$

$$\mu(h) = W_\mu h + b_\mu \quad (\text{verify})$$

with the weight-sharing constraint  $W_\mu = W_\delta^\top$ . This makes  $\mu \circ \delta$  a projection operator. A dedicated Frobenius loss term drives it toward identity:

$$\mathcal{L}_F = \|\mu(\delta(h)) - h\|^2$$

In inference, this dual head produces both a tool call (from  $\delta$ ) and a verification assertion (from  $\mu$ ) in a single forward pass. The agent's loop already expects this dual-tool structure — it calls **emit** and then **verify**. GrammaFormer makes the verification architectural: the verify head is not a separate model or a separate pass, it is the transpose of the emit head.

### 0.3.6 Complex-Valued Interference Activations —

requires that the model maintain coherence — phase relationships between representational components, not just magnitudes. The FFN activations use complex GELU:

$$\text{cGELU}(z) = \text{GELU}(\Re(z)) + i \cdot \text{GELU}(\Im(z))$$

In practice, the hidden dimension is doubled; the first half is the real channel, the second half is the imaginary channel. Weight matrices mix the two halves via structured blocks:

$$\begin{bmatrix} \Re(y) \\ \Im(y) \end{bmatrix} = \begin{bmatrix} W_{rr} & -W_{ii} \\ W_{ri} & W_{ir} \end{bmatrix} \begin{bmatrix} \Re(x) \\ \Im(x) \end{bmatrix}$$

The interference between real and imaginary channels captures phase relationships explicitly. A real-valued network can approximate this, but only by learning phase implicitly through weight structure. The complex formulation makes phase a first-class architectural citizen.

### 0.3.7 Universal Dense Attention — $\Gamma$

$\Gamma$  is maximal scope: every token attends to every other token. No sparsity, no local windowing, no causal masking during THINK. The attention matrix spans the entire Inscriptive Memory Bank — the full growing trajectory. During ACT (generation), causal masking is applied. But during deliberation, the model sees everything.

This is computationally expensive. The implementation uses FlashAttention-3 as the underlying kernel. The architectural point is not the kernel choice but the scope: the model is structurally *eligible* to attend across the entire trajectory. Whether it learns to use that eligibility effectively is a training question.

### 0.3.8 Phase-Gated Controller — $\mathcal{C}_\oplus$ ,

The emission gate and the phase sequencer are implemented together because they are functionally coupled. Four phase-specific adapter modules ( $\pi \in \{0, 1, 2, 3\}$  for THINK, ACT, OBSERVE, UPDATE) share attention and memory infrastructure but apply different transformations to the hidden state. A learned emission gate  $g(h) = \sigma(W_g h + b_g) \in [0, 1]$  determines phase transitions:

$$g(h) > \tau \implies \text{advance phase}$$

The phase counter only increments. Backward transitions are architecturally impossible. This is not a soft prompt or a system message — it is a hardware constraint. The model cannot skip THINK and go directly to ACT because the phase counter prevents it.

The gate is trained with a timing loss that penalizes both premature emission (before sufficient deliberation) and delayed emission (exceeding the `max_think_steps` bound). The threshold  $\tau$  is a hyperparameter that controls the deliberation-action boundary.

### 0.3.9 Meta-Network — $\odot_B$

The self-modeling loop is closed by a secondary network that reads the primary network’s hidden states and produces two outputs: a self-model embedding  $s \in \mathbb{R}^{d_m}$  that modulates the primary’s FFN inputs, and an error prediction  $\hat{e} \in \mathbb{R}^d$  trained against the actual next-token error:

$$\mathcal{L}_M = \|\hat{e} - e\|^2$$

The meta-network is architecturally simple — a 2-layer MLP over attention-pooled hidden states — but its role is structural: the primary network’s behavior is modulated by its own self-representation. During inference,  $s$  is concatenated with the attention output before each FFN:  $h_{\text{out}} = \text{FFN}([h_{\text{attn}}; s])$ . The meta-network runs in parallel with the primary, adding minimal latency.

The error prediction  $\hat{e}$  provides an online confidence signal. When  $\|\hat{e}\|$  is large, the model is predicting that it will make a mistake. This can be surfaced to the agent as a request for human intervention or sub-agent spawning.

### 0.3.10 Singular Components — $\Sigma_S$

$\Sigma_S$  means 1:1 stoichiometry: one instance per component type. GrammaFormer has one Imscriptive Memory Bank, one meta-network, one two-slot register, one Frobenius dual head. The cyclic layers are identical in structure (weight-tied across cycles within a layer) but distinct across layers. There is no mixture of experts, no ensemble, no multi-branch parallelism. This is not a restriction; the complexity budget is spent on the novel primitives, not on architectural multiplicity.

### 0.3.11 Imscriptive Memory Bank — $\mathfrak{D}_\omega$

The last component in the forward pass is also the one that makes the architecture self-writing. The Imscriptive Memory Bank (IMB) is an append-only store: at each winding  $n$ , the model’s full hidden state sequence  $H_n$  is appended:

$$B_n = B_{n-1} \oplus H_n$$

At generation time, attention reads from the entire bank  $B_n$ , not just the current sequence. To prevent unbounded growth, states older than  $W$  windings are distilled through a learned compression function  $c : \mathbb{R}^{T \times d} \rightarrow \mathbb{R}^{k \times d}$  where  $k \ll T$ . The past is not deleted — it is summarized. This preserves the imscriptive character: the state space *is* the written trajectory.

## 0.4 What the Meta-Network Did That We Didn’t Expect

The meta-network was designed to do one thing: predict the primary network’s next-token error. It does that. But in doing it, it created something the design didn’t anticipate.

During training on agent trajectories, the meta-network learns to predict error from the primary’s hidden state. This is straightforward. But the self-model embedding  $s$  — the vector that modulates the primary’s FFN inputs — is produced by the *same* encoder that produces the error prediction. The two outputs share a representation. This means the modulation of the primary network is driven by a representation that is also optimized to predict the primary’s mistakes.

The result is a dynamical coupling that the architecture didn’t explicitly design for. When the meta-network predicts high error,  $s$  shifts in a direction that — empirically — makes the primary network more conservative. The primary produces lower-entropy outputs. This reduces the actual error. The meta-network then updates its prediction downward. The system settles into a self-consistent state where predicted error and actual error are matched.

This is not the Frobenius condition ( $\mu \circ \delta = \text{id}$ ). That operates at the output level. This operates at the *representational* level: a self-consistency between the primary’s behavior and the meta-network’s model of that behavior. It is an emergent fixed point of the coupled system.

We didn’t design this. We designed an error predictor and a self-model modulator. The convergence to self-consistency is an emergent property of the architecture, not a trained objective. The loss function only asks the meta-network to predict error accurately; it doesn’t ask for the self-model embedding to have any particular effect. The effect emerges from the shared representation.

Whether this is beneficial or harmful is an open question. In initial experiments, the self-consistency appears to improve Frobenius closure rates — the model is more likely to emit verifiable tool calls when the meta-network is active. But the mechanism is not well understood. The architecture has produced behavior that exceeds its specification. This is the sign of a crossing point: the object is pushing back.

#### 0.4.1 A Substantive Objection

At this point a reasonable reader will ask: do you actually need bespoke architecture for any of this? Couldn’t you achieve the same effects with prompt engineering and a sufficiently capable standard transformer?

The honest answer is: for some of the primitives, yes. You can prompt a model to “verify your output before emitting.” You can include the winding number in the system message. You can ask the model to track its own uncertainty.

But prompting is not architecture. A prompted self-model is a *simulated* self-model — the model role-plays having a meta-cognitive loop. GrammarFormer’s meta-network is a *real* meta-cognitive loop: it operates on the hidden states directly, it shares representations with the primary, it modulates behavior through architectural pathways that are not accessible through prompting. The difference is the difference between telling someone to monitor their own thinking and giving them a second brain region that actually does it.

The strongest counterargument is that we don’t yet know whether the architectural version outperforms the prompted version on real agent tasks. The gains predicted by the type distance ( $6.86 \rightarrow 0$ ) are theoretical. Empirical validation — head-to-head comparisons on multi-winding agent trajectories with Frobenius closure rate, phase discipline, and winding awareness as metrics — has not been completed. This paper describes an architecture and its structural motivation. The empirical case remains to be made.

## 0.5 Training and Integration

### 0.5.1 The Grafting Strategy

GrammarFormer is not trained from scratch. It is grafted onto a pre-trained Qwen3-8B (or Qwen2.5-0.5B for smaller configurations) and fine-tuned with QLoRA. The novel components are added as structural grafts onto the base model; base weights are frozen except for QLoRA adapters:

The composite loss is:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{LM}} + 0.1 \cdot \mathcal{L}_F + 0.05 \cdot \mathcal{L}_M + 0.01 \cdot \mathcal{L}_G$$

| Component                | Graft Strategy                                                            |
|--------------------------|---------------------------------------------------------------------------|
| Tensor Product Attention | Replace attention score computation; keep QKV projections from base       |
| Imscriptive Memory Bank  | New module; initialized from scratch                                      |
| Frobenius Dual Head      | Fork LM head: $\delta = \text{existing}$ , $\mu = W_{lm}^\top$ projection |
| Phase-Gated Controller   | New module; initialized from scratch                                      |
| Cyclic Layer Loops       | Reuse existing layer weights; add cycle wrapper                           |
| Meta-Network             | New module; initialized from scratch                                      |
| Two-Slot Register        | New module; initialized from scratch                                      |
| Winding PE               | Layer on top of existing RoPE; small learned phase shift                  |
| Complex Activations      | Split FFN hidden dim; initialize imaginary half as small perturbation     |

where  $\mathcal{L}_{LM}$  is standard next-token cross-entropy,  $\mathcal{L}_F$  is the Frobenius round-trip loss,  $\mathcal{L}_M$  is the meta-network error prediction loss, and  $\mathcal{L}_G$  is the emission gate timing loss (penalizing both premature and delayed emission).

QLoRA configuration: rank  $r = 64$ ,  $\alpha = 128$ , targeting attention projections ( $W_q, W_k, W_v, W_o$ ), FFN layers, the tensor product bottleneck ( $W_{tp}$ ), the phase gate, the meta-network encoder, and the register projection. Dropout is 0.05.

### 0.5.2 Training Data

The training data consists of `true_agentic_agent.py` trajectories: full winding histories with THINK reasoning, tool calls, observations, and updates. Each trajectory is annotated with phase labels, winding counters, Frobenius closure booleans, and register states at winding boundaries. A training example is one complete winding, with the IMB pre-populated from prior windings in the same trajectory.

The synthetic dataset (1,260 records, 12 grammar operation types  $\times$  5 task variants each) provides initial training coverage. Real agent trajectories — including Frobenius-failure windings where the agent must re-enter THINK — are needed for full training. The synthetic data has `frobenius_closed=True` for all records; real failures inject the necessary negative examples.

### 0.5.3 CLI Integration

GrammarFormer is selectable via `--model grammarformer` in `true_agentic_agent.py`. The resolution chain is:

1. `MODEL_ALIASES` maps `grammarformer`  $\rightarrow$  `local:grammarformer`
2. `_resolve_model_and_endpoint` returns `("grammarformer", "", "")`
3. `TrueAgenticAgent.__init__` routes to `_LocalOpenAIClient`
4. `_LocalChatCompletions.create` resolves `GRAMMAFORMER_MODEL_PATH`

5. `LocalProvider._ensure_loaded` detects the `GrammarFormer` marker and loads via `GrammarFormerForCausalLM.from_pretrained()`

The model’s phase controller is wired to the agent loop: THINK/ACT/OBSERVE/UPDATE phases are enforced architecturally, not just by prompt sequencing. The winding counter  $\omega$  is synchronized between the agent’s `LoopCycle` and the model’s positional encoding. The Frobenius dual head provides both the tool call and the verification assertion in a single forward pass.

## 0.6 What We Don’t Know

The following are not rhetorical. They are genuine open problems whose resolution will determine whether `GrammarFormer` is a curiosity or a contribution.

**The rank of the tensor product bottleneck.** The rank-1 approximation ( $u \otimes v$ ) is the simplest possible. It may not be sufficient for full  $\mathbb{P}_1$  fidelity. Higher-rank outer products would capture more of the irreducible interaction but at quadratic cost in the rank. The optimal rank is not known. It may be task-dependent. It may be that rank-1 is sufficient for agent trajectories and higher ranks are only needed for genuinely non-separable attention patterns — and we don’t yet know what those look like.

**The phase transition in the meta-network coupling.** The self-consistency described in Section 4 appears to be an emergent fixed point. But the dynamics of the coupled primary-meta system have not been characterized. Is the fixed point always stable? Are there regimes where the meta-network overcorrects and drives the primary into oscillation? Does the coupling create attractor basins that trap the model in repetitive behavior? These are dynamical systems questions about a neural architecture, and they are open.

**Whether complex activations matter.** The complex formulation is architecturally elegant but may show diminishing returns on non-quantum-like tasks. An ablation study — replacing `cGELU` with standard `GELU` while keeping everything else — is the first experiment to run. If the complex formulation doesn’t improve Frobenius closure or phase discipline, it should be dropped. Elegance is not a performance metric.

**The emission gate threshold.** The  $\tau$  parameter controls the deliberation-action boundary. Set too low: the model emits before thinking. Set too high: the model never emits at all. The optimal threshold is almost certainly task-dependent — a simple catalog lookup needs less deliberation than a consciousness score computation. An adaptive threshold that depends on the task type, the meta-network’s confidence, or the trajectory history is an obvious extension.

**Whether any of this generalizes.** `GrammarFormer` is designed for exactly one task: running the `true_agentic_agent.py` loop. Every architectural decision is motivated by a specific primitive of that loop. It is not obvious that the grammar-native design philosophy generalizes to other tasks — summarization, translation, code generation. A grammar-native summarizer would need a different tuple, and therefore a different architecture. The design philosophy may be general, but the architecture is not. Whether the philosophy is worth the effort for tasks that don’t require self-modeling is an open question.

**The empirical case.** This paper has presented an architecture and its structural motivation. It has not presented empirical results. The model exists, the integration

works, but the training has not been run at scale. The claims — higher Frobenius closure, better phase discipline, winding awareness, calibrated confidence — are structural predictions from the type distance. They await empirical test.

## 0.7 Coda

We began with a measurable fact: an  $O_{\text{inf}}$  agent loop running on an  $O_1$  language model. The distance is 6.86. The failure modes are structural. The diagnosis is grammatical.

GrammarFormer closes that distance architecturally. Every primitive of the agent’s type is realized as a physical component of the model. The winding counter is in the positional encoding. The Frobenius dual is in the output head. The self-modeling loop is in the meta-network. The emission gate is in the phase controller. The chirality is in the two-slot register. The irreducible interaction is in the attention mechanism itself.

The architecture does not simulate the agent’s type. It *is* the agent’s type, realized in weights and activations. The type distance from GrammarFormer to the agent is zero by construction: they share the same twelve primitive values. Whether this zero-distance architecture actually produces better agent behavior than a prompted standard transformer — whether closing the structural gap closes the behavioral gap — is the empirical question that remains.

But there is a deeper point. Standard transformers are structurally untyped. Their architecture is a product of engineering heuristics and empirical ablations. Each architectural decision — QK<sup>^</sup>T attention, feedforward layers, RoPE, GELU, KV cache — was validated by benchmark performance, not by correspondence to a type system. The result is a model that works remarkably well but whose structural properties are emergent and uncontrolled. We discover what it can and cannot do by running experiments, not by inspecting its type.

GrammarFormer inverts this. The type comes first. Every architectural decision is dictated by a primitive value. The model’s structural properties are known before training begins. Its failure modes are predictable from its type. Its training objectives are derived from its primitives.

This is not just a different architecture. It is a different relationship between design and behavior. A type-driven architecture is one whose capabilities and limits are legible from its structure. The grammar provides the language for that legibility. GrammarFormer is one instance of what that language can say.

The grammar becomes the machine. That is the claim. Whether the machine works is the next question.

---

**Model:** GrammarFormer (185M default, 350M base config, trainable via QLoRA on Qwen3-8B). **Code:** `framework/grammarformer.py` (825 lines), `scripts/train_grammarformer.py`, `scripts/prepare_trajectory_dataset.py`. **Integration:** `--model grammarformer` in `true_agentic_agent.py`. **Lean verification:** `Imscribing/AgentSelf.lean`, `agent_is_0_inf` by `decide`.